

19 More Bayesian Analysis of a Numerical Variable

Example 19.1. How late do people arrive at parties? [FiveThirtyEight](#) conducted a survey to address this question. We'll assume this is a reasonably representative sample and arrival times are measured reliably. Arrival times are measured in minutes after the scheduled start time, rounded to the nearest minute. (Negative arrival times represent arrivals before the scheduled start time.)

1. What would you expect the population distribution of party arrival times to look like? For example, what percent of arrivals would you expect to be early (negative)? Later than two hours? What would you expect of other features like shape, center, and variability?
2. We will start by assuming that, given the values of relevant parameters, arrival times follow a conditional Normal distribution. Describe what this assumption says about arrival times. What are the relevant parameters, and what do they represent? (We will revisit this conditional Normal assumption later, but go with it for now.)
3. Suggest a prior distribution. Explain your reasoning.
4. Describe how you could simulate the prior predictive distribution.

5. This [applet](#) conducts the simulation from the previous part. Move the sliders to enter your prior mean and SD for μ and σ . Does the distribution of arrival times seem reasonable based on your expectations? For example, what percent of arrivals are early (negative), or later than two hours — and do these values seem reasonable? If not, revise your assumptions and try again (i.e., play with the sliders in the applet) until you find a distribution that is reasonable. Do not worry about getting it perfect; you just want to settle on assumptions that provide a reasonable starting point. (The assumptions include both the conditional Normal model for arrival times and the prior distribution of parameters. For now we are focusing on the prior distribution, so take the conditional Normal model as given. We'll revisit that assumption later.)

Example 19.2. Section [19.1.2](#) summarizes the FiveThirtyEight sample¹ of 803 arrival times (rounded to the nearest minute).

We should always first explore the sample data, but to illustrate a point we will not do that yet. Instead, we'll skip straight to fitting the model.

1. Use `brms` to fit the model and summarize the posterior distribution. Briefly describe the posterior distribution.
2. Simulate the posterior predictive distribution. Briefly describe the posterior predictive distribution.

¹We couldn't find the raw FiveThirtyEight data online, so this data was constructed to match the histogram in the article.

3. Perform a visual posterior predictive check. What does the posterior predictive check say about the appropriateness of our model? Could any problems be fixed by switching from a Normal to a t likelihood?

Example 19.3. The sample data suggests a model that allows for skewness would be more appropriate than Normal. This primarily results in a change of the likelihood function. There are several likelihood functions we can use, but we'll try a [Skew Normal](#) family, which introduces an additional skewness parameter α . Since α is a parameter it also needs a prior distribution. Rather than redoing our prior predictive tuning, we'll rely on the fact that software packages like `brms` can utilize default priors.

1. Use `brms` to fit the skew Normal model, letting `brms` choose the prior for α (but specifying the priors for μ and σ as before). What prior does `brms` use?
2. Summarize and describe the posterior distribution.
3. Use simulation to approximate the posterior predictive distribution. Summarize and describe the posterior predictive distribution.
4. Perform a visual posterior predictive check. What does the posterior predictive check say about the appropriateness of the model? Which model seems to be a better fit for the data: likelihood based on Normal or Skew-Normal?

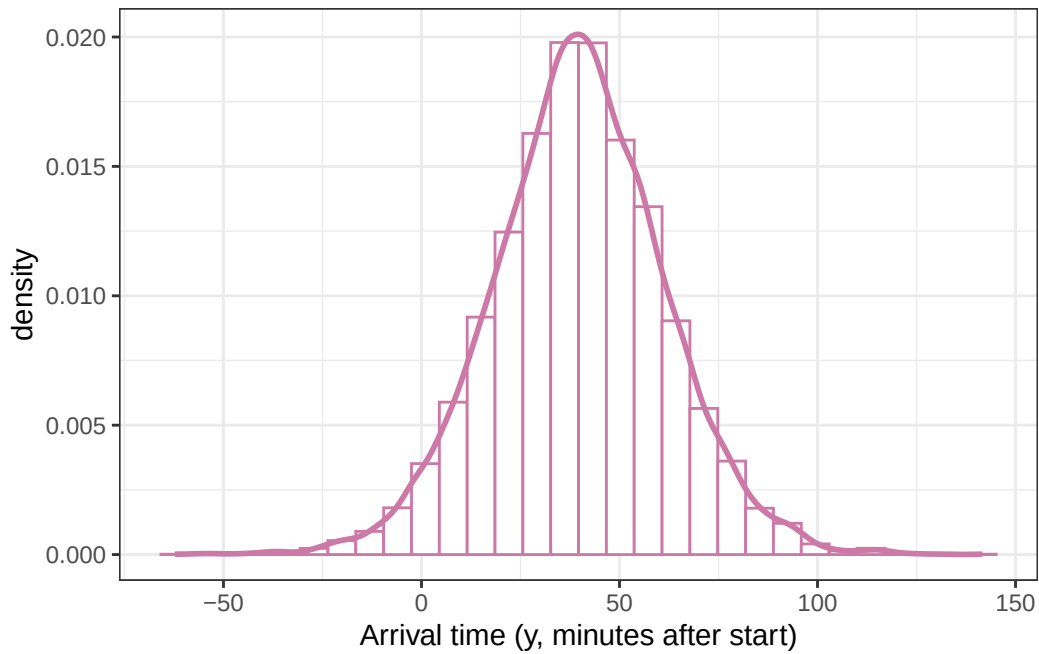
5. Fit the model to the data in `brms` using the `skew_normal` family but letting `brms` choose the prior for all parameters. What prior does `brms` choose? Is the posterior sensitive to the choice of prior?
6. Are the posterior distributions of parameters skewed or relatively symmetric? Is the posterior predictive distribution skewed or relatively symmetric?
7. Write a few sentences summarizing the conclusions of the Bayesian model in context.

19.1 Notes

19.1.1 Prior predictive distribution (Normal likelihood)

```
n_rep = 10000  
  
mu_sim = rnorm(n_rep, 40, sd = 7)  
  
sigma_sim = pmax(0.001, rnorm(n_rep, 20, 5))  
  
y_sim = rnorm(n_rep, mu_sim, sigma_sim)  
  
ggplot(data.frame(y_sim),  
       aes(x = y_sim)) +
```

```
geom_histogram(aes(y = after_stat(density)),
               col = bayes_col["prior_predict"],
               fill = "white") +
geom_density(col = bayes_col["prior_predict"],
             linewidth = 1) +
labs(x = "Arrival time (y, minutes after start)") +
theme_bw()
```



19.1.2 Sample data

```
data = read_csv("party-time.csv")
```

Rows: 803 Columns: 1

-- Column specification -----

Delimiter: ","

dbl (1): minutes

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
head(data)
```

```
# A tibble: 6 x 1
  minutes
  <dbl>
1     -10
2     -12
3     -18
4     -21
5     -15
6      -9
```

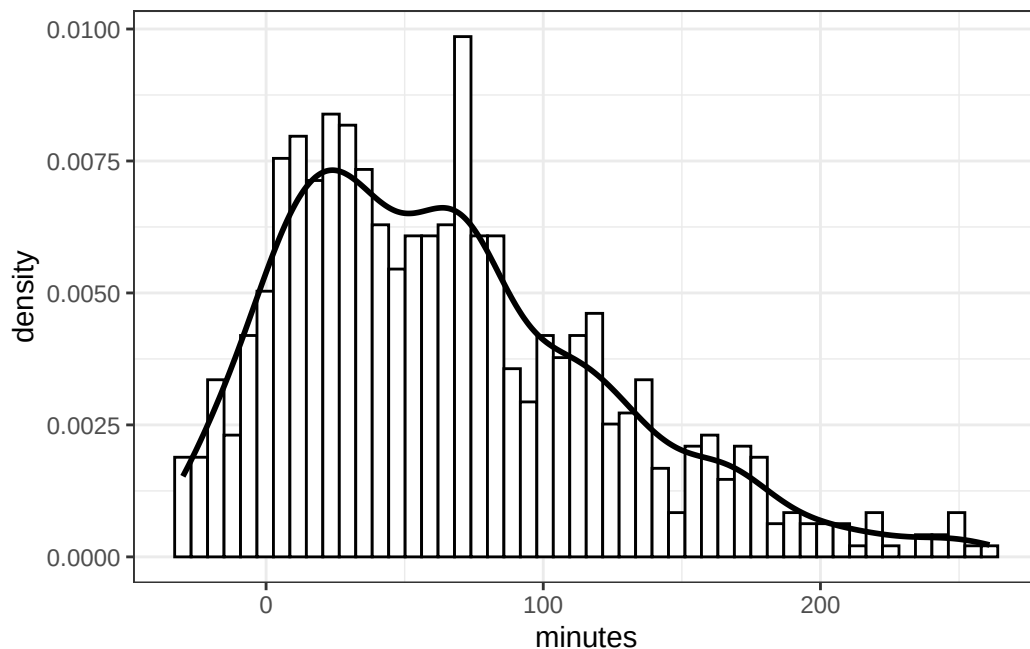
```
summary(data$minutes)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-30.00	21.00	58.00	65.48	101.00	261.00

```
sd(data$minutes)
```

```
[1] 58.20041
```

```
ggplot(data,
  aes(x = minutes)) +
  geom_histogram(aes(y = after_stat(density)),
    bins = 50,
    col = "black", fill = "white") +
  geom_density(linewidth = 1) +
  theme_bw()
```



19.1.3 Posterior distribution (Normal likelihood)

```
library(brms)
```

Loading required package: Rcpp

Loading 'brms' package (version 2.23.0). Useful instructions can be found by typing `help('brms')`. A more detailed introduction to the package is available through `vignette('brms_overview')`.

Attaching package: 'brms'

The following objects are masked from 'package:tidybayes':

`dstudent_t`, `pstudent_t`, `qstudent_t`, `rstudent_t`

The following object is masked from 'package:stats':

`ar`

```
library(tidybayes)
library(bayesplot)
```

This is bayesplot version 1.14.0

- Online documentation and vignettes at mc-stan.org/bayesplot
- bayesplot theme set to bayesplot::theme_default()
 - * Does `_not_` affect other ggplot2 plots
 - * See `?bayesplot_theme_set` for details on theme setting

Attaching package: 'bayesplot'

The following object is masked from 'package:brms':

`rhat`

```
fit <- brm(data = data,
  family = gaussian,
  minutes ~ 1,
  prior = c(prior(normal(40, 7), class = Intercept),
    prior(normal(20, 5), class = sigma)),
  iter = 3500,
  warmup = 1000,
  chains = 4,
  refresh = 0)
```

Compiling Stan program...

Start sampling

```
summary(fit)
```



```

Family: gaussian
Links: mu = identity
Formula: minutes ~ 1
Data: data (Number of observations: 803)
Draws: 4 chains, each with iter = 3500; warmup = 1000; thin = 1;
       total post-warmup draws = 10000

```

Regression Coefficients:

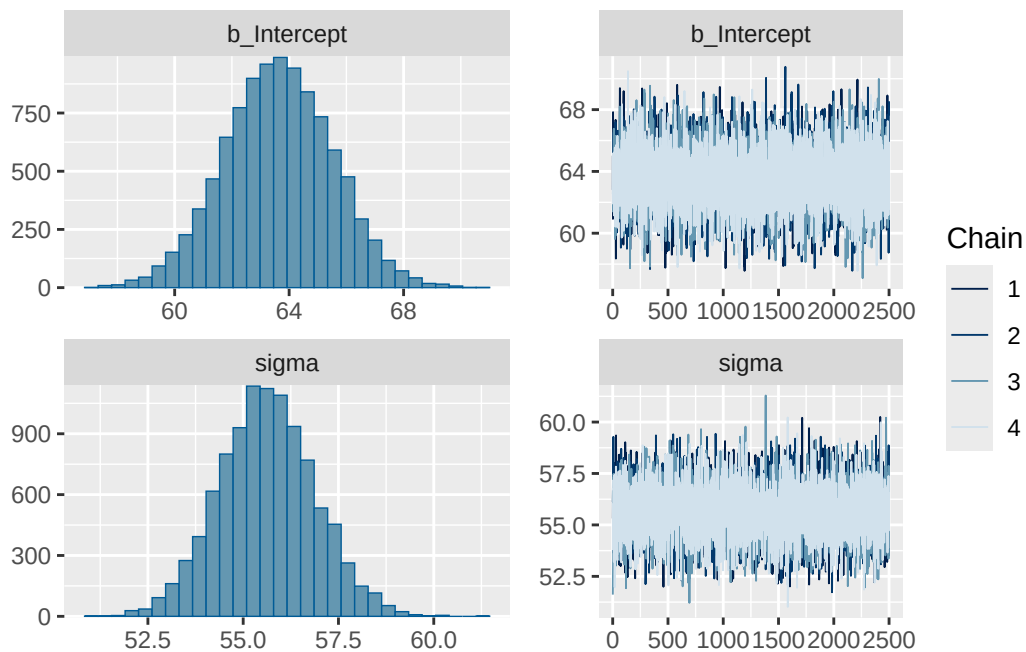
	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	63.62	1.90	59.89	67.31	1.00	7925	6536

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	55.63	1.26	53.19	58.16	1.00	8029	5966

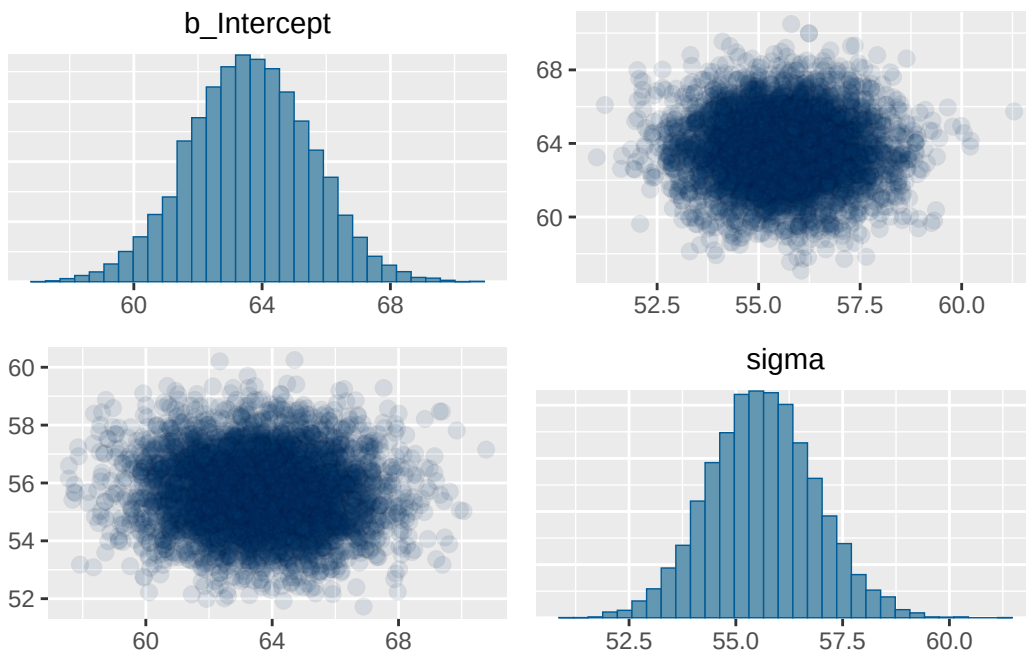
Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

```
plot(fit)
```



.chain	.iteration	.draw	mu	sigma
1	1	1	65.02298	55.31455
1	2	2	60.92569	57.07551
1	3	3	67.84054	55.26640
1	4	4	64.99232	54.94991
1	5	5	62.73370	56.72687
1	6	6	63.22659	56.69350
1	7	7	64.23976	54.95613
1	8	8	64.38425	54.61429
1	9	9	61.26003	55.34205
1	10	10	61.87223	56.32483

```
pairs(fit,
      off_diag_args = list(alpha = 0.1))
```



```
posterior = fit |>
  spread_draws(b_Intercept, sigma) |>
  rename(mu = b_Intercept)

posterior |> head(10) |> kbl() |> kable_styling()
```

.chain	.iteration	.draw	mu	sigma	y_sim
1	1	1	65.02298	55.31455	63.927464
1	2	2	60.92569	57.07551	24.429415
1	3	3	67.84054	55.26640	115.485023
1	4	4	64.99232	54.94991	-9.153019
1	5	5	62.73370	56.72687	70.348386
1	6	6	63.22659	56.69350	70.117555
1	7	7	64.23976	54.95613	46.968762
1	8	8	64.38425	54.61429	10.376603
1	9	9	61.26003	55.34205	39.564128
1	10	10	61.87223	56.32483	94.967298

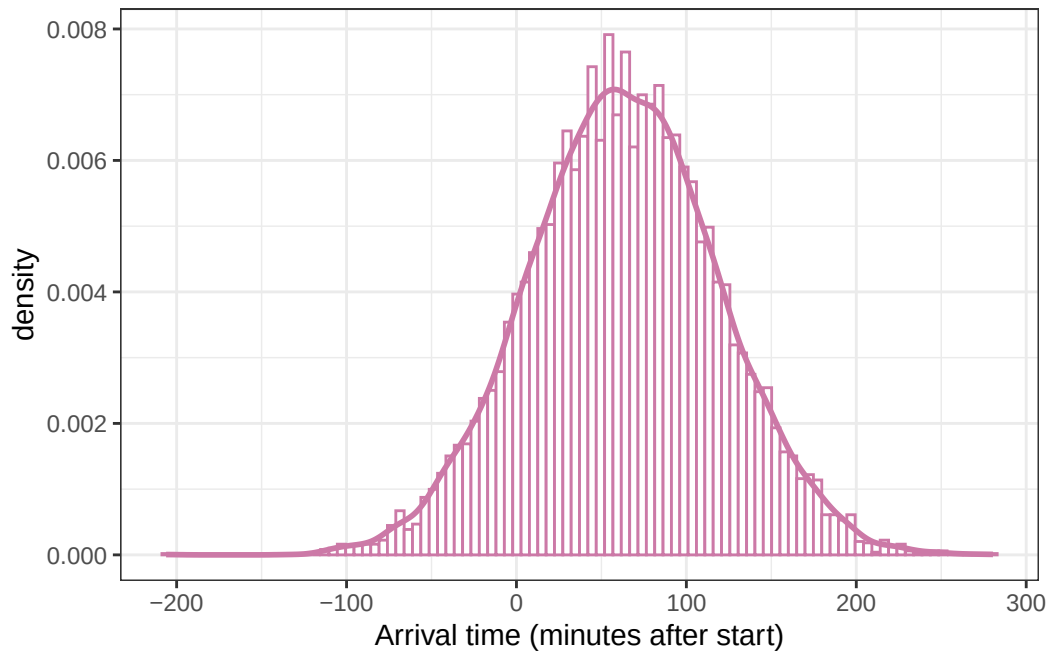
19.1.4 Posterior predictive distribution (Normal likelihood)

```
n_rep = nrow(posterior)

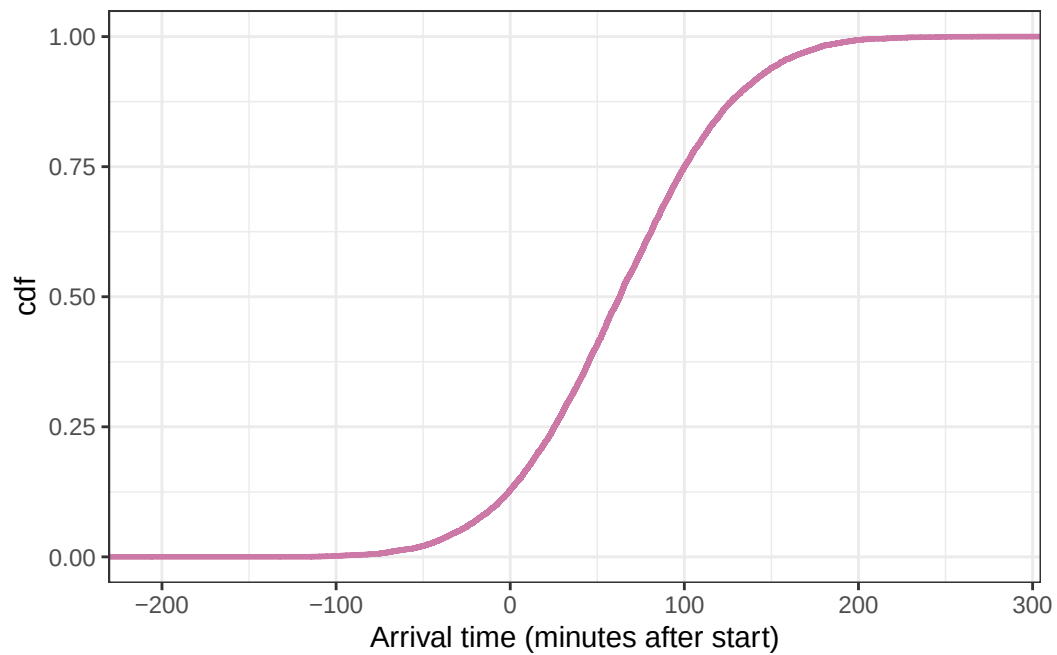
posterior <- posterior |>
  mutate(y_sim = rnorm(n_rep,
                        mean = posterior$mu,
                        sd = posterior$sigma))

posterior |> head(10) |> kbl() |> kable_styling()
```

```
ggplot(posterior,
       aes(x = y_sim)) +
  geom_histogram(aes(y = after_stat(density)),
                 col = bayes_col["posterior_predict"], fill = "white", bins = 100) +
  geom_density(linewidth = 1, col = bayes_col["posterior_predict"]) +
  labs(x = "Arrival time (minutes after start)") +
  theme_bw()
```



```
# plot the cdf
ggplot(posterior,
       aes(x = y_sim)) +
  stat_ecdf(col = bayes_col["prior_predict"],
           linewidth = 1) +
  labs(x = "Arrival time (minutes after start)",
       y = "cdf") +
  theme_bw()
```

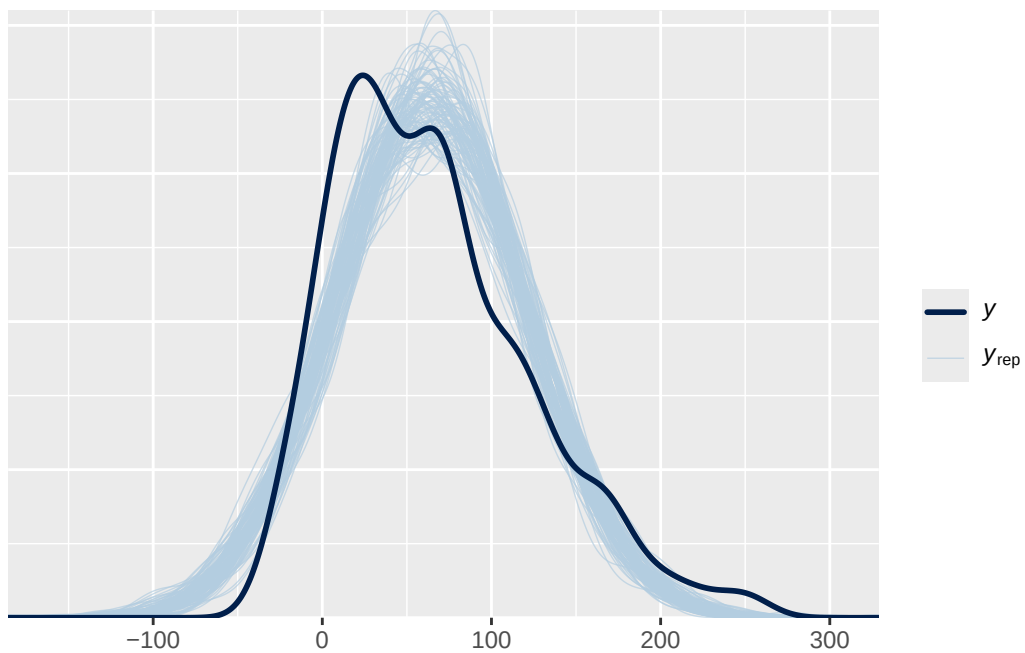


```
quantile(posterior$y_sim, c(0.005, 0.995))
```

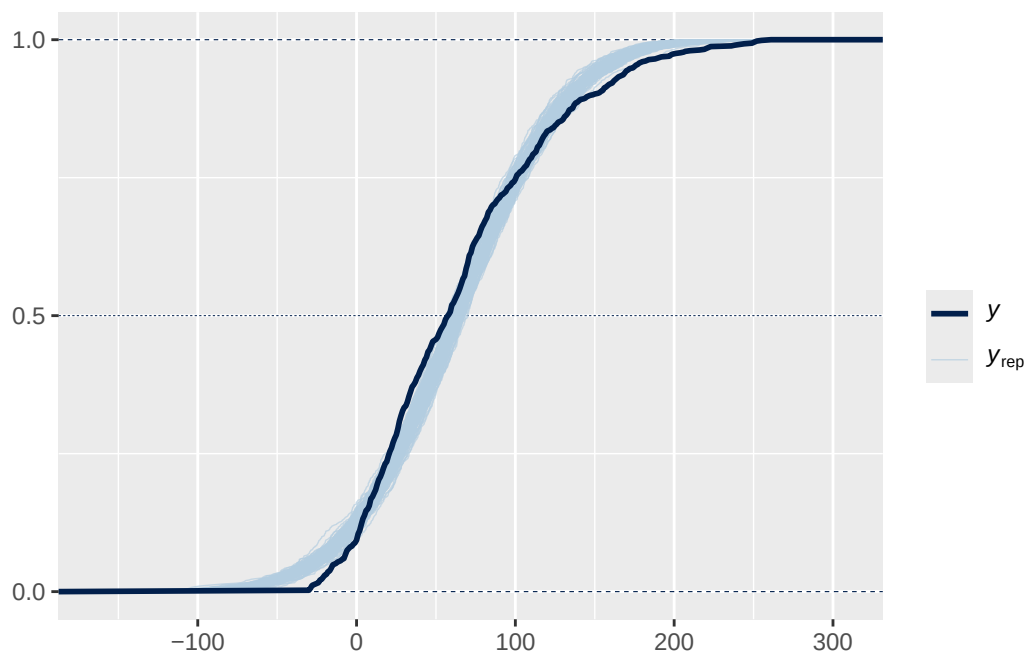
```
      0.5%      99.5%  
-80.72404 205.14377
```

19.1.5 Posterior predictive checking (Normal likelihood)

```
pp_check(fit, ndraw = 100)
```



```
pp_check(fit, ndraw = 100, type = "ecdf_overlay")
```



19.1.6 Posterior distribution (Skew-Normal likelihood)

```
fit <- brm(data = data,
  family = skew_normal,
  minutes ~ 1,
  prior = c(prior(normal(40, 7), class = Intercept),
    prior(normal(20, 5), class = sigma)),
  iter = 3500,
  warmup = 1000,
  chains = 4,
  refresh = 0)
```

Compiling Stan program...

Start sampling

```
prior_summary(fit)
```

	prior	class	coef	group	resp	dpar	nlpar	lb	ub	source	tag
	normal(0, 4)	alpha								default	
	normal(40, 7)	Intercept								user	
	normal(20, 5)	sigma						0		user	

```
summary(fit)
```

Family: skew_normal
Links: mu = identity
Formula: minutes ~ 1
Data: data (Number of observations: 803)
Draws: 4 chains, each with iter = 3500; warmup = 1000; thin = 1;
total post-warmup draws = 10000

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	61.30	1.76	57.90	64.75	1.00	6030	6494

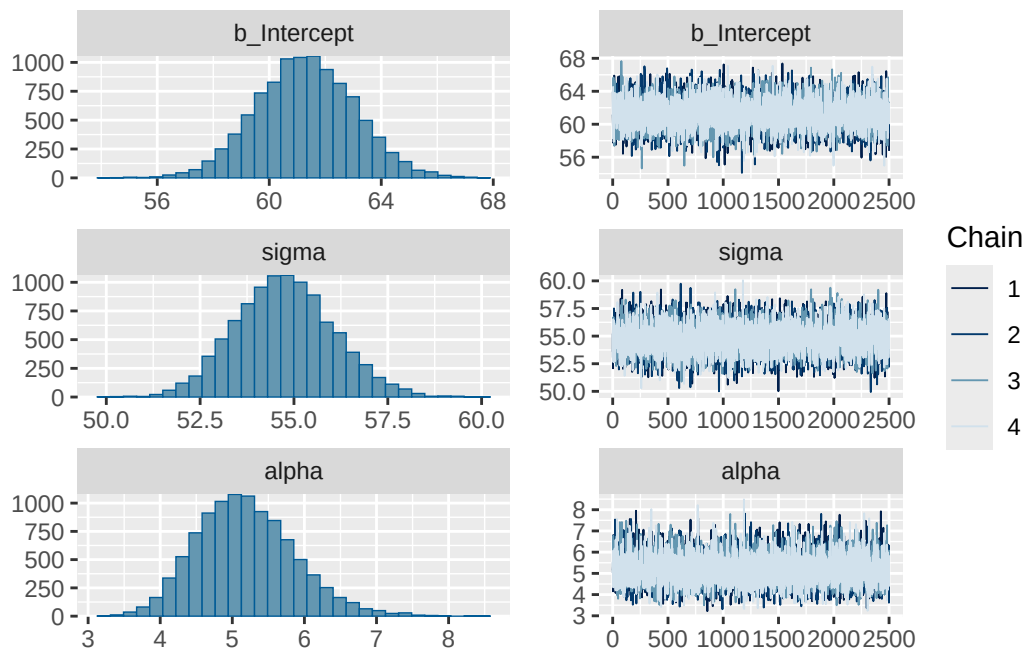
Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	54.75	1.31	52.27	57.40	1.00	5397	6267

alpha	5.19	0.68	3.99	6.66	1.00	6015	5680
-------	------	------	------	------	------	------	------

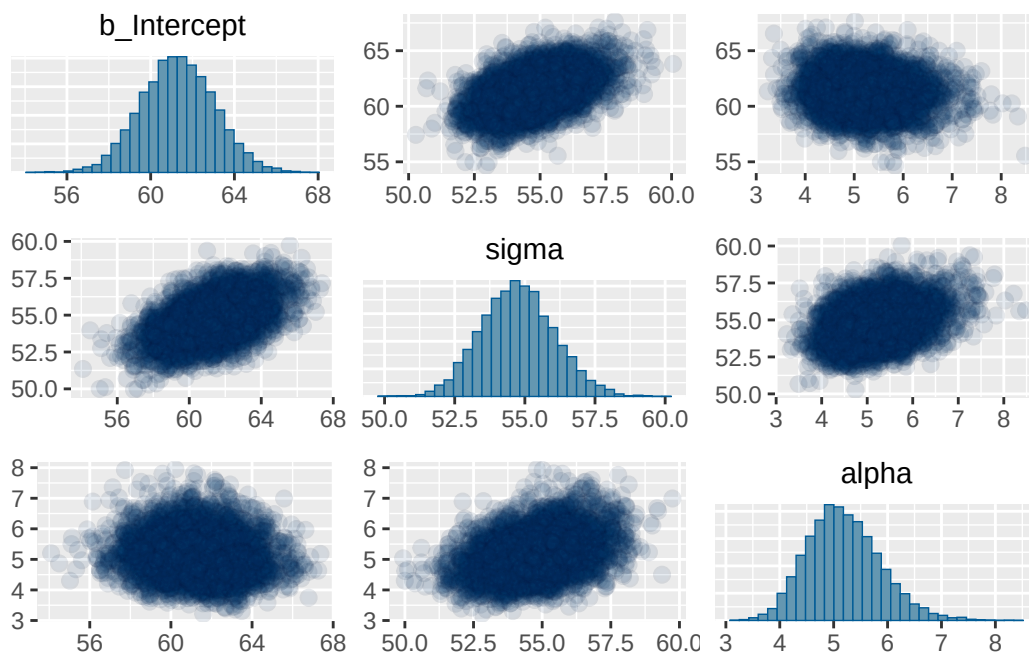
Draws were sampled using `sampling(NUTS)`. For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

```
plot(fit)
```



```
pairs(fit,
      off_diag_args = list(alpha = 0.1))
```


.chain	.iteration	.draw	mu	sigma	alpha
1	1	1	61.14092	53.58990	5.141311
1	2	2	59.82107	55.95199	5.327699
1	3	3	61.04911	52.08854	4.153378
1	4	4	58.84429	55.56491	5.909929
1	5	5	59.31336	54.05318	6.806539
1	6	6	57.72592	55.37278	5.847652
1	7	7	60.52559	54.32741	5.343750
1	8	8	59.47829	55.14924	5.466411
1	9	9	61.26679	53.03494	5.763313
1	10	10	59.42221	54.83531	5.288244



```
posterior = fit |>
  spread_draws(b_Intercept, sigma, alpha) |>
  rename(mu = b_Intercept)

posterior |> head(10) |> kbl() |> kable_styling()
```

19.1.7 Posterior predictive distribution (Skew-Normal likelihood)

`posterior_predict` simulates a sample of size 803 for each of the 10000 simulated (μ, σ, α) triples from the posterior distribution, resulting in a 10000 x 803 matrix.

```
y_predict = posterior_predict(fit)

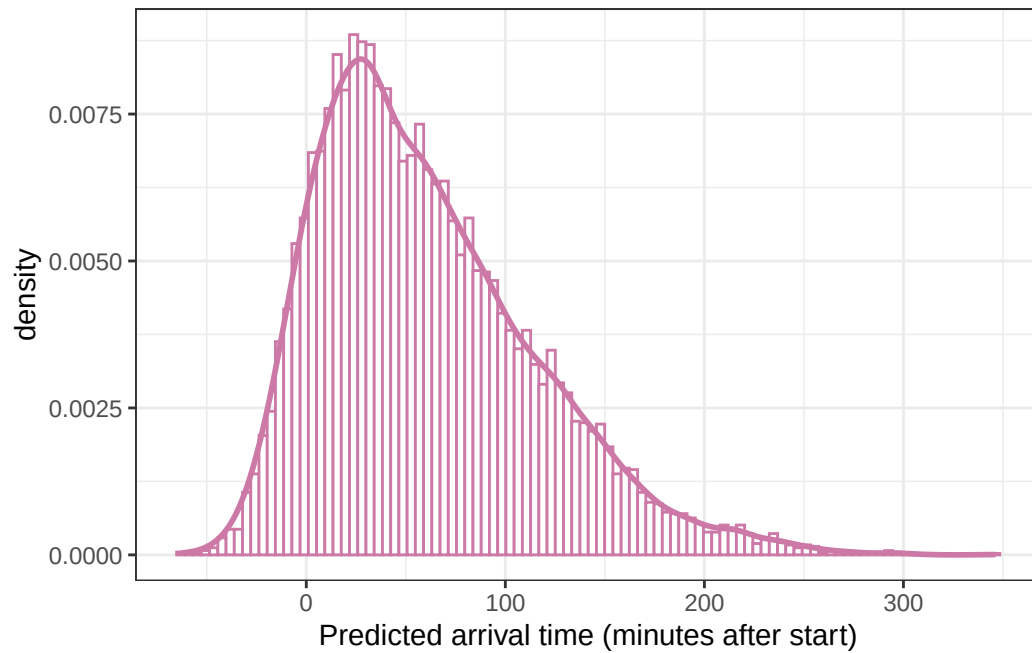
y_predict[1:5, 1:5]
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	95.76681	52.87998	19.36111	67.996502	55.8700750
[2,]	78.47290	129.68699	46.94753	238.718608	-0.9809806
[3,]	110.50418	17.12719	152.60890	21.829781	-0.4454336
[4,]	136.53967	98.71127	67.16218	-3.115918	69.4129385
[5,]	121.13833	80.83180	141.74894	69.710591	18.4639068

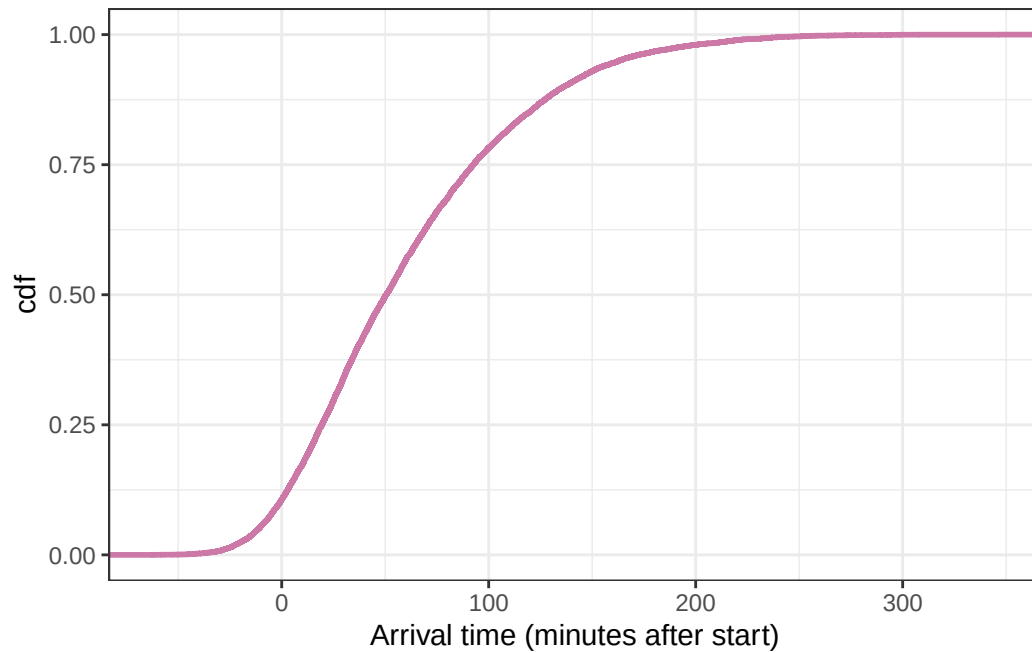
We'll just select the first column to get 10000 simulated y values.

```
y_predict = data.frame(y_sim = y_predict[, 1])
```

```
ggplot(y_predict,
       aes(x = y_sim)) +
  geom_histogram(aes(y = after_stat(density)),
                col = bayes_col["posterior_predict"], fill = "white", bins = 100) +
  geom_density(linewidth = 1, col = bayes_col["posterior_predict"]) +
  labs(x = "Predicted arrival time (minutes after start)") +
  theme_bw()
```



```
# plot the cdf
ggplot(y_predict,
       aes(x = y_sim)) +
  stat_ecdf(col = bayes_col["prior_predict"],
           linewidth = 1) +
  labs(x = "Arrival time (minutes after start)",
       y = "cdf") +
  theme_bw()
```

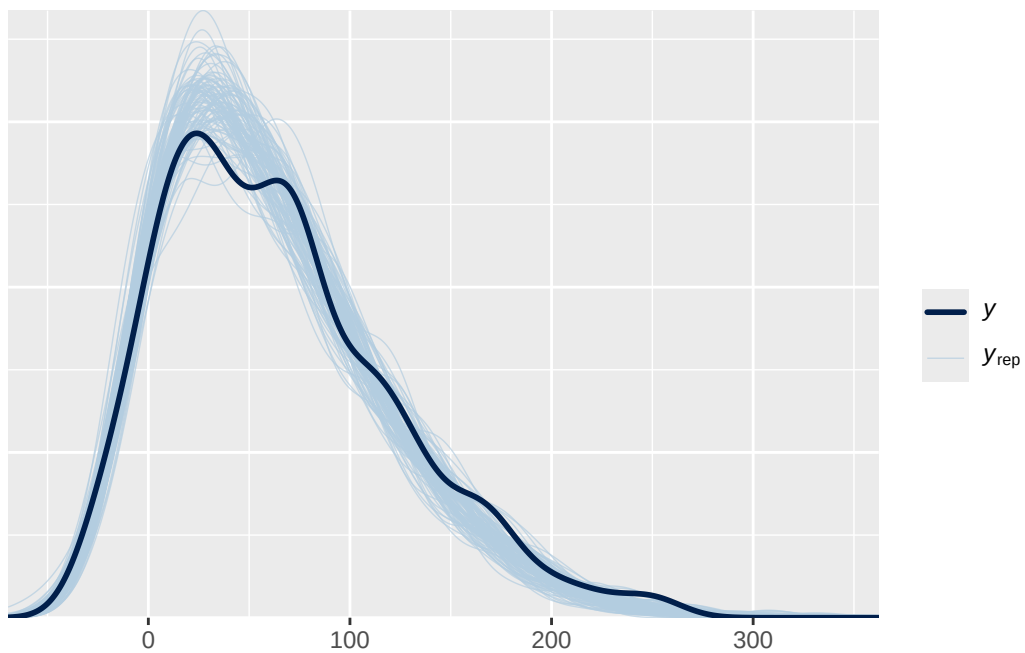


```
quantile(y_predict$y_sim, c(0.005, 0.995))
```

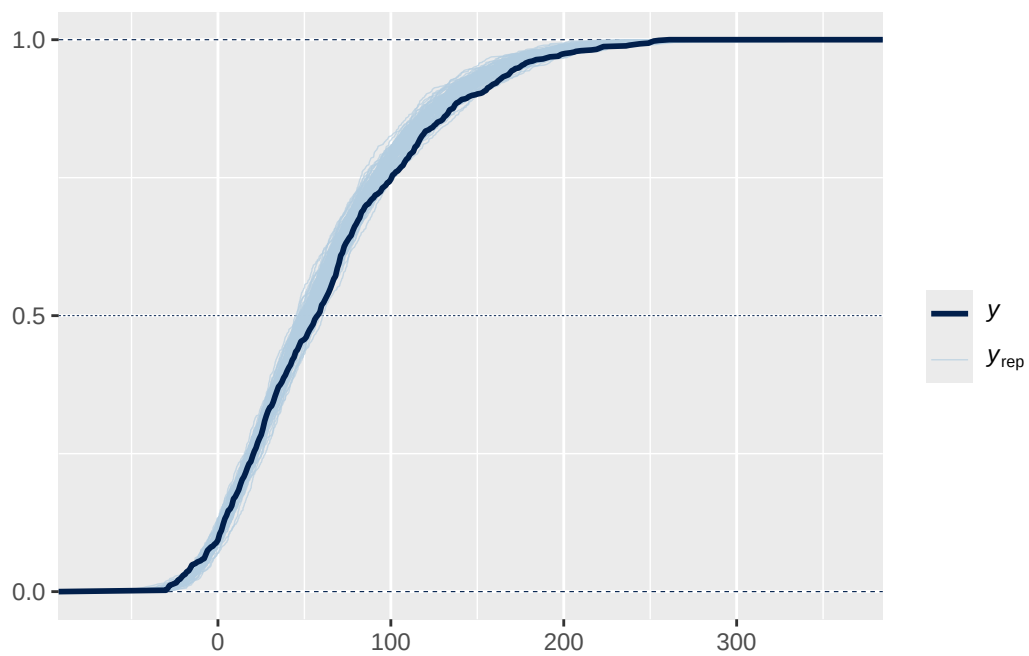
```
      0.5%      99.5%  
-33.97992 239.61983
```

19.1.8 Posterior predictive checking (Skew-Normal likelihood)

```
pp_check(fit, ndraw = 100)
```



```
pp_check(fit, ndraw = 100, type = "ecdf_overlay")
```



19.1.9 Default brms priors (Skew-Normal likelihood)

```
fit <- brm(data = data,  
  family = skew_normal(),  
  minutes ~ 1,  
  iter = 3500,  
  warmup = 1000,  
  chains = 4,  
  refresh = 0)
```

Compiling Stan program...

Start sampling

```
prior_summary(fit)
```

	prior	class	coef	group	resp	dpar	nlpar	lb	ub	source	tag
	normal(0, 4)	alpha								default	
student_t(3, 58, 57.8)	Intercept									default	
student_t(3, 0, 57.8)	sigma							0		default	

```
summary(fit)
```

Family: skew_normal
Links: mu = identity
Formula: minutes ~ 1
Data: data (Number of observations: 803)
Draws: 4 chains, each with iter = 3500; warmup = 1000; thin = 1;
total post-warmup draws = 10000

Regression Coefficients:

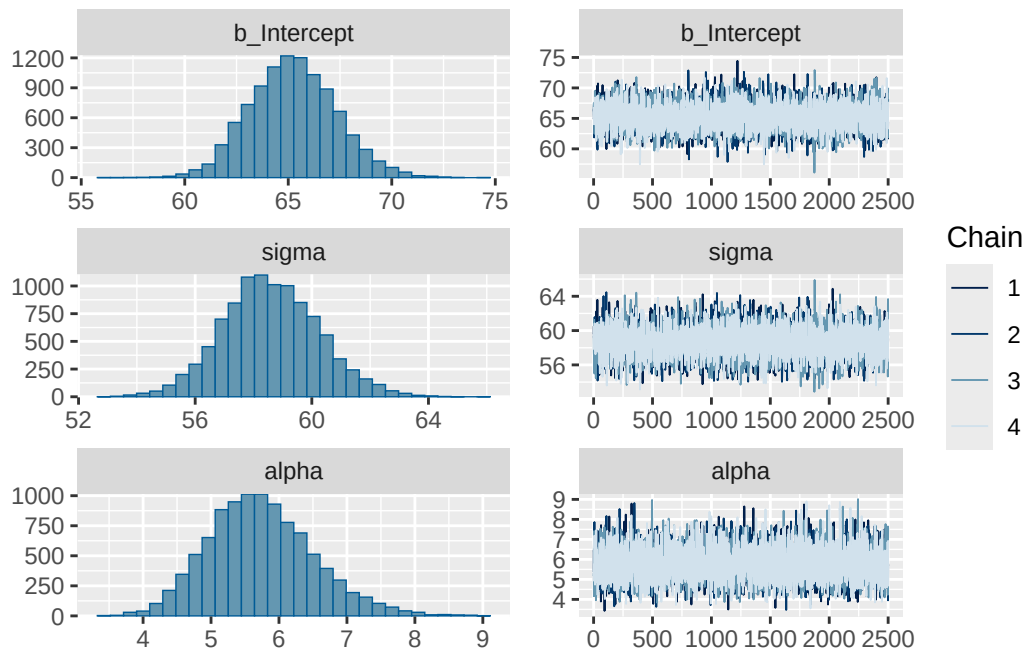
	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	65.23	2.08	61.37	69.47	1.00	5282	4964

Further Distributional Parameters:

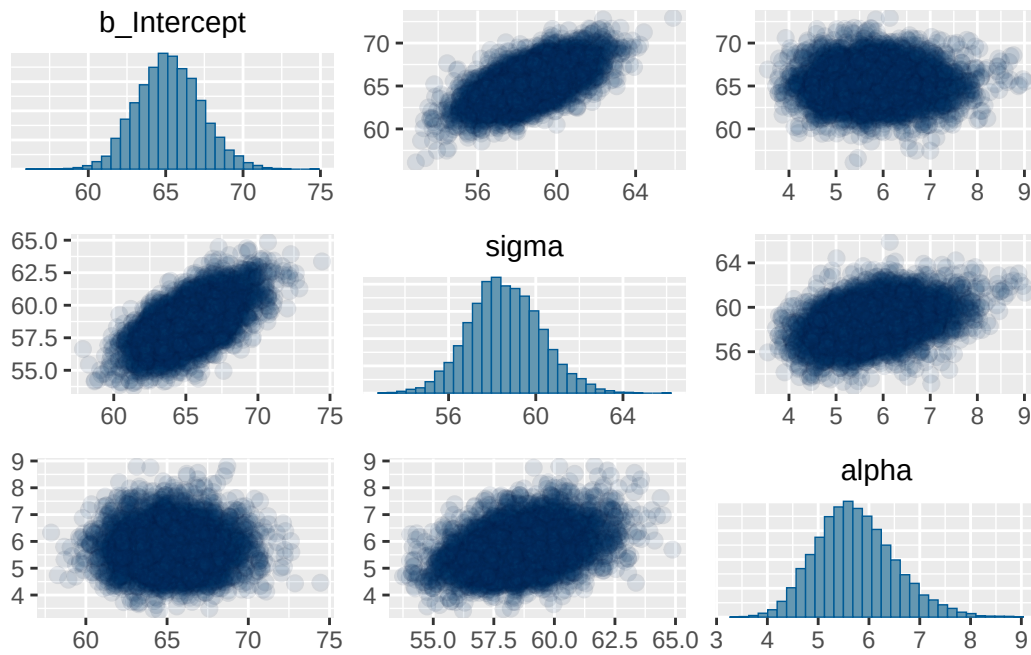
	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	58.59	1.65	55.45	61.99	1.00	5142	5357
alpha	5.74	0.78	4.37	7.43	1.00	5634	4362

Draws were sampled using `sampling(NUTS)`. For each parameter, `Bulk_ESS` and `Tail_ESS` are effective sample size measures, and `Rhat` is the potential scale reduction factor on split chains (at convergence, `Rhat` = 1).

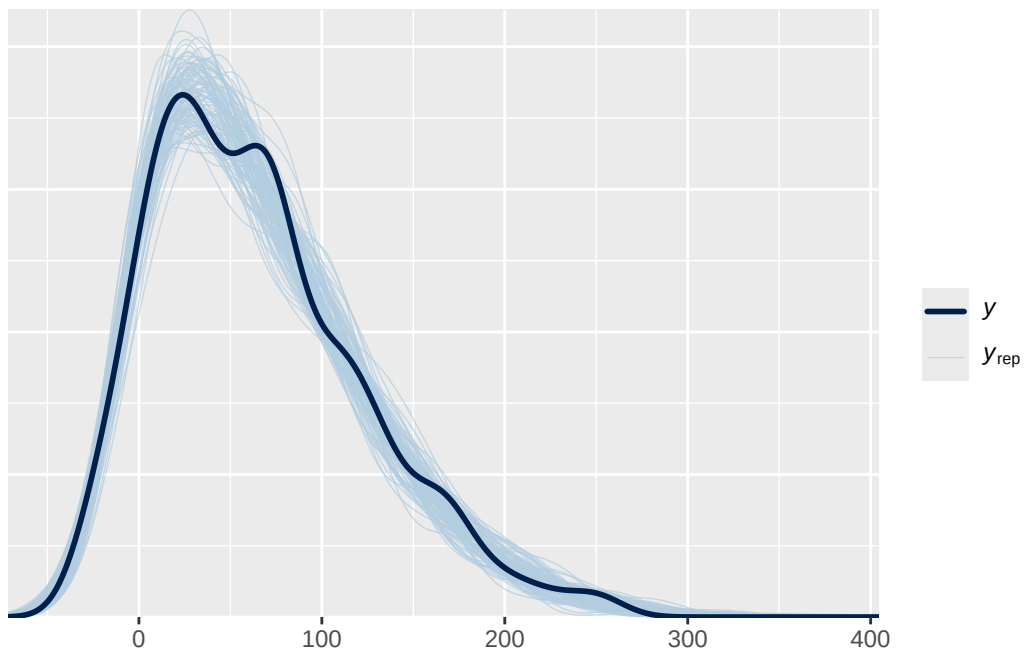
```
plot(fit)
```



```
pairs(fit,
      off_diag_args = list(alpha = 0.1))
```



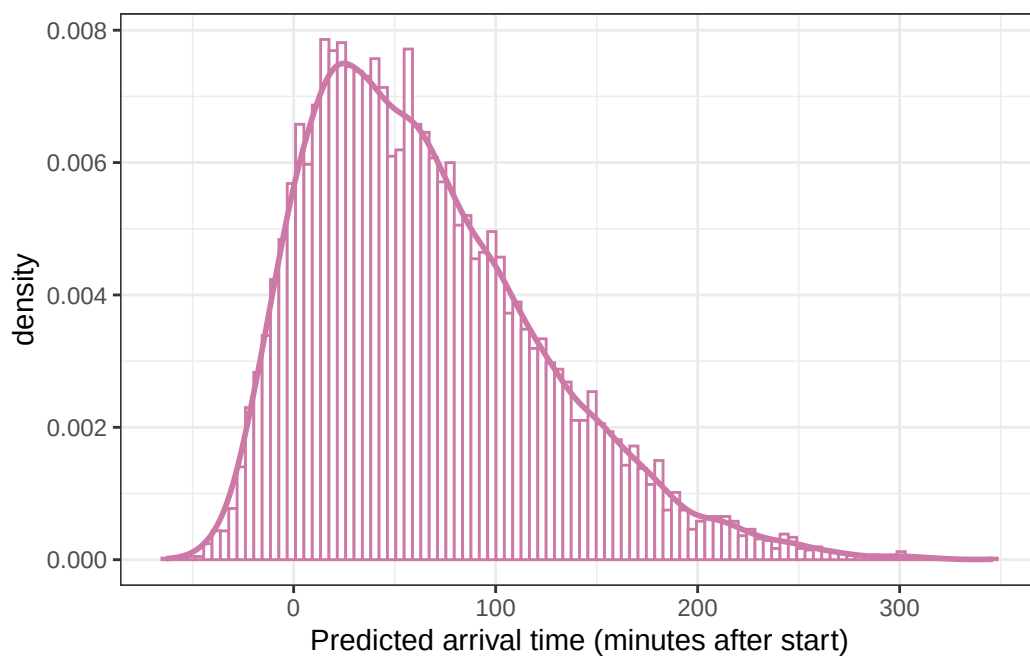
```
pp_check(fit, ndraw = 100)
```



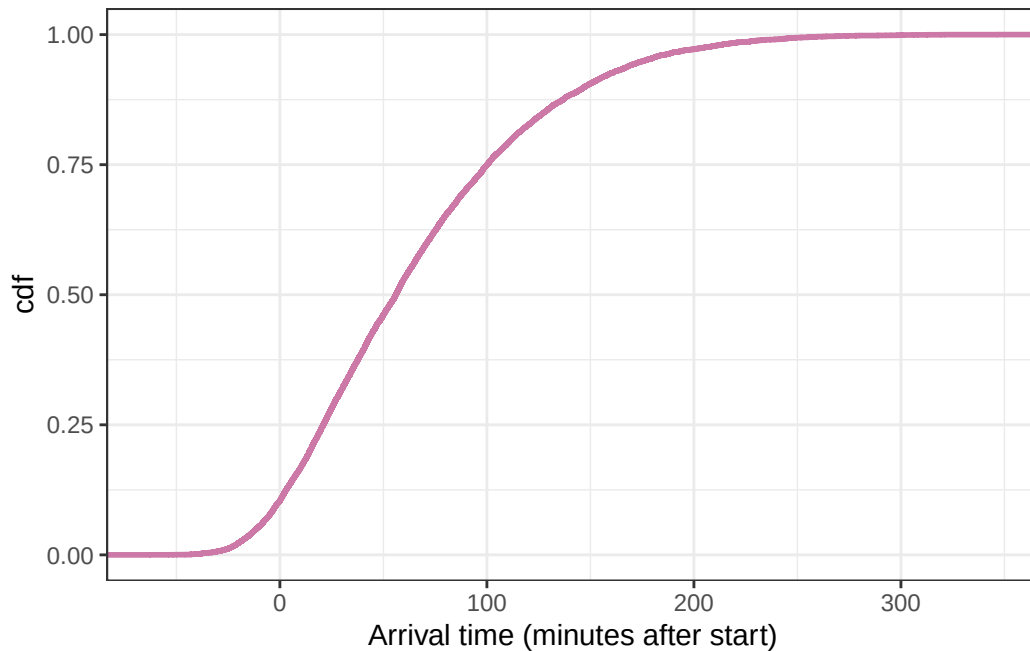

```
y_predict = posterior_predict(fit)

y_predict = data.frame(y_sim = y_predict[, 1])
```

```
ggplot(y_predict,
       aes(x = y_sim)) +
  geom_histogram(aes(y = after_stat(density)),
                col = bayes_col["posterior_predict"], fill = "white", bins = 100) +
  geom_density(linewidth = 1, col = bayes_col["posterior_predict"]) +
  labs(x = "Predicted arrival time (minutes after start)") +
  theme_bw()
```



```
# plot the cdf
ggplot(y_predict,
       aes(x = y_sim)) +
  stat_ecdf(col = bayes_col["prior_predict"],
            linewidth = 1) +
  labs(x = "Arrival time (minutes after start)",
       y = "cdf") +
  theme_bw()
```



```
quantile(y_predict$y_sim, c(0.005, 0.995))
```

```
      0.5%      99.5%
-32.32382 255.38655
```

19.1.10 Code for Shiny app

Here is some code for creating a rough [applet](#) that you can embed within an RMarkdown file by adding `runtime: shiny` to the YAML metadata; see [Shiny Documents](#). (This code is not evaluated; you would need to run it on your own.)

```
sliderInput("mu_prior_mean", "Prior mean of mu:", value = 10, min = -60, max = 120)

sliderInput("mu_prior_sd", "Prior SD of mu:", value = 10, min = 5, max = 120)

sliderInput("sigma_prior_mean", "Prior mean of sigma:", value = 10, min = 5, max = 120)

sliderInput("sigma_prior_sd", "Prior SD of sigma:", value = 10, min = 5, max = 120)

renderPlot({
```

```

n_rep = 10000

sim_prior = data.frame(
  mu = rnorm(n_rep, input$mu_prior_mean, input$mu_prior_sd),
  sigma = rgamma(n_rep,
    shape = input$sigma_prior_mean ^ 2 / input$sigma_prior_sd ^ 2,
    rate = input$sigma_prior_mean / input$sigma_prior_sd ^ 2)) %>%
  mutate(y_predict = rnorm(n_rep, mu, sigma))

p1 = sim_prior %>%
  ggplot(aes(x = y_predict)) +
  geom_density(col = bayes_col[4]) +
  labs(x = "Arrival time (minutes)",
    title = "Simulated prior predictive distribution of arrival times: Density") +
  scale_x_continuous(breaks = seq(-60, 180, by = 30),
    limits = c(-60, 180)) +
  theme_bw()

p2 = sim_prior %>%
  ggplot(aes(x = y_predict)) +
  stat_ecdf(col = bayes_col[4]) +
  labs(x = "Arrival time (minutes)",
    y = "Cumulative probability",
    title = "Simulated prior predictive distribution of arrival times: CDF") +
  scale_x_continuous(breaks = seq(-60, 180, by = 30),
    limits = c(-60, 180)) +
  theme_bw()

grid.arrange(p1, p2)

})

```